

Diseño y fragmentación de una base de datos distribuida

Práctica de Fragmentación Distribuida — Sistema de Cafetería Multi-Sede

Jeremy Ruperto Gaibor Rodriguez

Ingeniería de Software — Aplicaciones Distribuidas (ISR-701)
Universidad Técnica Estatal de Quevedo (UTEQ)

8 de julio de 2026

1. Introducción

Las bases de datos distribuidas permiten repartir la información entre distintos nodos físicos sin perder la posibilidad de consultarla como si fuera un único sistema [1]. En esta práctica se diseñó un esquema de fragmentación horizontal, vertical y mixta para el sistema de pedidos de una cafetería con tres sedes: Campus, Babahoyo y Ventanas. El objetivo fue distribuir la información de clientes y pedidos entre tres nodos PostgreSQL independientes, verificar que la fragmentación cumple las propiedades de completitud, reconstrucción y disjunción, y construir vistas globales capaces de reconstruir la información como si estuviera centralizada, apoyándose en el módulo `postgres_fdw`.

Este informe documenta el entorno desplegado, las decisiones de diseño tomadas para cada tipo de fragmentación, la evidencia de ejecución obtenida mediante `psql`, y una reflexión final sobre las ventajas y dificultades observadas durante el desarrollo.

2. Entorno operativo

El entorno se construyó con `docker-compose` [3], levantando tres contenedores PostgreSQL independientes, uno por cada sede de la cafetería (Campus, Babahoyo, Ventanas), cada uno con su propio volumen de datos y su propio puerto expuesto en el host (5433, 5434 y 5435 respectivamente). Esta separación de puertos y volúmenes garantiza que cada nodo es completamente autónomo: no comparte disco ni proceso con los demás.

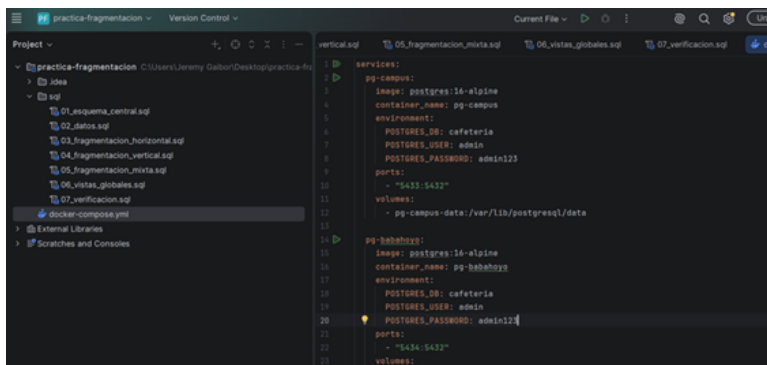


Figura 1: Definición de los tres contenedores PostgreSQL en `docker-compose.yml`.

Tras ejecutar `docker compose up -d`, los tres contenedores quedaron en ejecución simultánea, lo que se confirmó con `docker compose ps` (Figura 2).

La conexión a cada nodo se realizó con el cliente `psql` incluido en la propia imagen del contenedor, sin necesidad de instalar herramientas adicionales en el host:

```
1 docker exec -it pg-campus psql -U admin -d cafeteria
```

3. Esquema centralizado

Antes de fragmentar la base de datos fue necesario definir el esquema completo en un único nodo de referencia (`pg-campus`). Este esquema centralizado contiene las tablas `clientes`, `productos` y `pedidos`, con sus respectivas claves primarias y las claves foráneas que relacionan los pedidos con clientes y productos.

```
1 CREATE TABLE clientes (  
2     cliente_id SERIAL PRIMARY KEY,
```

```
PS C:\Users\Jeremy.Saibor\Desktop\practica-fragmentacion> docker compose up -d
[+] up 14/14
✓ Image postgres:16-alpine Pulled
failed to do request: GET https://registry-1.docker.io/v2/libra/p2/postgres/referenc/sha256:7648773567161032787561163567972efadeaf2720ac3117935316
ended

What's next?
During this Compose error with Gordon > docker at "help me fix this compose error"
Filter, search, and stream logs from all your Compose services
In one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs

PS C:\Users\Jeremy.Saibor\Desktop\practica-fragmentacion> docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
PS C:\Users\Jeremy.Saibor\Desktop\practica-fragmentacion> docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS NAMES
PS C:\Users\Jeremy.Saibor\Desktop\practica-fragmentacion> docker compose up -d
[+] up 7/7
✓ Network practica-fragmentacion_default Created
✓ Volume practica-fragmentacion-pg-ventanas-data Created
✓ Volume practica-fragmentacion-pg-campus-data Created
✓ Volume practica-fragmentacion-pg-babehoyo-data Created
✓ Container pg-campus Started
✓ Container pg-babehoyo Started
✓ Container pg-ventanas Started

What's next?
Filter, search, and stream logs from all your Compose services
In one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs

PS C:\Users\Jeremy.Saibor\Desktop\practica-fragmentacion> docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
pg-babehoyo postgres:16-alpine "docker-entrypoint.sh" pg-babehoyo 15 seconds ago Up 14 seconds 0.0.0.0:5435->5432/tcp, :::5435->5432/tcp
pg-campus postgres:16-alpine "docker-entrypoint.sh" pg-campus 15 seconds ago Up 14 seconds 0.0.0.0:5433->5432/tcp, :::5433->5432/tcp
pg-ventanas postgres:16-alpine "docker-entrypoint.sh" pg-ventanas 15 seconds ago Up 14 seconds 0.0.0.0:5439->5432/tcp, :::5435->5432/tcp
PS C:\Users\Jeremy.Saibor\Desktop\practica-fragmentacion>
```

Figura 2: Los tres nodos levantados en segundo plano con un solo comando.

```
PS C:\Users\Jeremy Gaibor\Desktop\practica-fragmentacion> docker exec -it pg-campus psql -U admin -d cafeteria
psql (16.14)
Type "help" for help.

cafeteria=#
```

Figura 3: Sesión `psql` abierta dentro del contenedor `pg-campus`.

```

3 nombre VARCHAR(80) NOT NULL,
4 ciudad VARCHAR(40) NOT NULL,
5 email VARCHAR(120) NOT NULL,
6 telefono VARCHAR(20)
7 );
8
9 CREATE TABLE productos (
10 producto_id SERIAL PRIMARY KEY,
11 nombre VARCHAR(80) NOT NULL,
12 categoria VARCHAR(30) NOT NULL,
13 precio NUMERIC(6,2) NOT NULL CHECK (precio >= 0)
14 );
15
16 CREATE TABLE pedidos (
17 pedido_id SERIAL PRIMARY KEY,
18 cliente_id INTEGER NOT NULL REFERENCES clientes(cliente_id),
19 producto_id INTEGER NOT NULL REFERENCES productos(producto_id),
20 fecha DATE NOT NULL,
21 monto NUMERIC(8,2) NOT NULL CHECK (monto >= 0),
22 sede VARCHAR(20) NOT NULL
23 );

```

El script se aplicó con:

```
1 docker exec -i pg-campus psql -U admin -d cafeteria < sql/01_esquema_central.sql
```

```
C:\Users\Jeremy Gaibor\Desktop\practica-fragmentacion>docker exec -i pg-campus psql -U admin -d cafeteria < sql/01_esque
ma_central.sql
CREATE TABLE
CREATE TABLE
CREATE TABLE

C:\Users\Jeremy Gaibor\Desktop\practica-fragmentacion>
```

Figura 4: Ejecución sin errores de 01_esquema_central.sql en pg-campus.

Posteriormente se cargaron los datos de prueba (6 clientes, 5 productos y 8 pedidos) mediante `sql/02_datos.sql`, que sirvieron como base para todo el proceso de fragmentación posterior.

4. Fragmentación horizontal

La tabla **pedidos** se fragmentó horizontalmente usando la columna **sede** como criterio de partición: cada nodo recibe únicamente las filas cuya sede coincide con la sede física del contenedor. De los 8 pedidos totales, 3 corresponden a Campus, 3 a Babahoyo y 2 a Ventanas.

```

1  -- Ejemplo para pg-babahoyo: solo se insertan los pedidos con sede='Babahoyo'
2  CREATE TABLE pedidos (
3      pedido_id    INTEGER PRIMARY KEY,
4      cliente_id   INTEGER NOT NULL,
5      producto_id  INTEGER NOT NULL,
6      fecha        DATE NOT NULL,
7      monto        NUMERIC(8,2) NOT NULL,
8      sede         VARCHAR(20) NOT NULL
9  );
10
11  INSERT INTO pedidos VALUES
12      (2, 2, 3, '2026-03-01', 2.50, 'Babahoyo'),
13      (6, 6, 1, '2026-03-03', 0.75, 'Babahoyo'),
14      (8, 2, 2, '2026-03-04', 1.00, 'Babahoyo');

```

Este mismo patrón se repitió en **pg-campus** y **pg-ventanas**, filtrando cada vez por su propia sede. Con esto se garantiza **disjunción** (ningún pedido aparece en dos nodos) y, al sumar las filas de los tres nodos, se recupera el total original, lo que garantiza **completitud**.

5. Fragmentación vertical

Para la fragmentación vertical se dividió la tabla **clientes** en dos fragmentos según la sensibilidad de la información:

- **clientes_publicos** (**cliente_id**, **nombre**, **ciudad**) alojado en **pg-campus**.
- **clientes_contacto** (**cliente_id**, **email**, **telefono**) alojado en **pg-babahoyo**.

En ambos fragmentos se conserva **cliente_id** como clave primaria replicada, lo que permite reconstruir cualquier registro completo mediante un **JOIN** por esa clave.

```
C:\Users\Jeremy Gaibor\Desktop\practica-fragmentacion>docker exec -i pg-campus psql -U admin -d cafeteria < sql\02_datos.sql
INSERT 0 6
INSERT 0 5
INSERT 0 8
C:\Users\Jeremy Gaibor\Desktop\practica-fragmentacion\
```

Figura 5: Creación y carga de **clientes_publicos** en **pg-campus**.

```
C:\Users\Jeremy Gaibor\Desktop\practica-fragmentacion>docker exec -i pg-babahoyo psql -U admin -d cafeteria -c "CREATE TABLE clientes_contacto (cliente_id INTEGER PRIMARY KEY, email VARCHAR(120) NOT NULL, telefono VARCHAR(20)); INSERT INTO clientes_contacto VALUES (1, 'maria@uteq.edu.ec', '0991111111'), (2, 'luis@uteq.edu.ec', '0992222222'), (3, 'ana@uteq.edu.ec', '0993333333'), (4, 'jose@uteq.edu.ec', '0994444444'), (5, 'carla@uteq.edu.ec', '0995555555'), (6, 'pedro@uteq.edu.ec', '0996666666');"
CREATE TABLE
INSERT 0 6

C:\Users\Jeremy Gaibor\Desktop\practica-fragmentacion>docker exec -it pg-babahoyo psql -U admin -d cafeteria -c "SELECT * FROM clientes_contacto;"
 cliente_id | email | telefono
-----
 1 | maria@uteq.edu.ec | 0991111111
 2 | luis@uteq.edu.ec | 0992222222
 3 | ana@uteq.edu.ec | 0993333333
 4 | jose@uteq.edu.ec | 0994444444
 5 | carla@uteq.edu.ec | 0995555555
 6 | pedro@uteq.edu.ec | 0996666666
(6 rows)
```

Figura 6: Creación y carga de **clientes_contacto** en **pg-babahoyo**.

6. Fragmentación mixta

El diseño mixto combina ambas técnicas sobre la tabla **clientes**: primero se particiona horizontalmente por ciudad (Quevedo frente a Babahoyo/Ventanas) y, dentro de cada partición, se vuelve a dividir verticalmente entre datos públicos y datos de contacto. El resultado se resume en la Tabla 1.

Cuadro 1: Distribución de los fragmentos mixtos de **clientes** por nodo.

Bloque	Nodo	Fragmento	Filas (cliente_id)
1	pg-campus	clientes_publicos_quevedo	1, 3
2	pg-babahoyo	clientes_contacto_quevedo	1, 3
3	pg-ventanas	clientes_publicos_otros	2, 4, 5, 6
4	pg-babahoyo	clientes_contacto_otros	2, 4, 5, 6

La Figura 7 adapta el esquema de fragmentación mixta a los datos reales manejados en esta práctica.

Cada fragmento cumple completitud (todo cliente aparece en exactamente un bloque público y un bloque de contacto), reconstrucción (el **JOIN** por **cliente_id** entre bloque público y bloque de contacto reproduce la fila original) y disjunción (ningún cliente se repite dentro del mismo tipo de fragmento).

7. Vistas globales con **postgres_fdw**

Para poder consultar la información como si estuviera centralizada, se usó el módulo **postgres_fdw**, incluido de forma nativa en PostgreSQL, que permite que un nodo lea tablas de otro nodo como si fueran locales [2]. Desde **pg-campus** se definieron dos servidores foráneos (**srv_babahoyo** y **srv_ventanas**), sus respectivos *user mappings*, y tablas foráneas apuntando a **pedidos** en cada nodo remoto. Con ello se construyó la vista **pedidos_global** como la unión de los tres fragmentos horizontales:

```
1 CREATE VIEW pedidos_global AS
2 SELECT * FROM pedidos
3 UNION ALL
4 SELECT * FROM pedidos_babahoyo
5 UNION ALL
6 SELECT * FROM pedidos_ventanas;
```

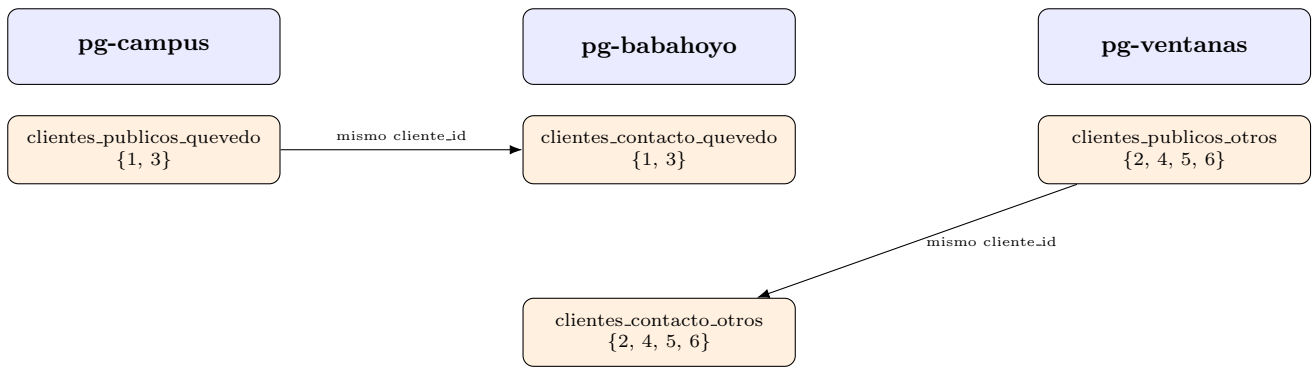


Figura 7: Fragmentación mixta aplicada a `clientes`: horizontal por ciudad (Quevedo vs. resto) y vertical por sensibilidad (público vs. contacto), con los identificadores reales de cliente en cada fragmento.

```
C:\Users\Jeremy Gaibor\Desktop\practica-fragmentacion>docker exec -it pg-ventanas psql -U admin -d cafeteria -c "SELECT
COUNT(*) FROM pedidos;"
count
-----
2
(1 row)

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image + docker debug pg-ventanas
Learn more at https://docs.docker.com/go/debug-cli/

C:\Users\Jeremy Gaibor\Desktop\practica-fragmentacion>docker exec -i pg-campus psql -U admin -d cafeteria -c "CREATE TAB
LE clientes_publicos_quevedo (cliente_id INTEGER PRIMARY KEY, nombre VARCHAR(80) NOT NULL, ciudad VARCHAR(40) NOT NULL);
INSERT INTO clientes_publicos_quevedo VALUES (1, 'Maria Alvarado', 'Quevedo'), (3, 'Ana Vera', 'Quevedo');"
CREATE TABLE
INSERT 0 2

C:\Users\Jeremy Gaibor\Desktop\practica-fragmentacion>docker exec -it pg-campus psql -U admin -d cafeteria -c "SELECT *
FROM clientes_publicos_quevedo;"
cliente_id | nombre | ciudad
-----
1 | Maria Alvarado | Quevedo
3 | Ana Vera | Quevedo
(2 rows)
```

Figura 8: Fragmento mixto de datos públicos de clientes de Quevedo en `pg-campus`.

```
C:\Users\Jeremy Gaibor\Desktop\practica-fragmentacion>docker exec -i pg-campus psql -U admin -d cafeteria < sql\06_vista
s_globales.sql
CREATE EXTENSION
CREATE SERVER
CREATE SERVER
CREATE USER MAPPING
CREATE USER MAPPING
CREATE FOREIGN TABLE
CREATE FOREIGN TABLE
CREATE VIEW
sede | total
-----
Campus | 1.00
Ventanas | 1.25
(2 rows)
```

Figura 9: Reconstrucción global de pedidos mediante `postgres_fdw` y `UNION ALL`.

De forma análoga se construyó `clientes_global`, uniendo por `cliente_id` los fragmentos públicos y de contacto de cada nodo para reconstruir el registro completo.

```
C:\Users\Jeremy Gaibor\Desktop\practica-fragmentacion>docker exec -i pg-campus psql -U admin -d cafeteria < sql\06_vista
s_globales.sql
NOTICE: extension "postgres_fdw" already exists, skipping
CREATE EXTENSION
ERROR: server "srv_babahoyo" already exists
ERROR: server "srv_ventanas" already exists
ERROR: user mapping for "admin" already exists for server "srv_babahoyo"
ERROR: user mapping for "admin" already exists for server "srv_ventanas"
ERROR: relation "pedidos_babahoyo" already exists
ERROR: relation "pedidos_ventanas" already exists
ERROR: relation "pedidos_global" already exists
sede | total
-----
Campus | 1.00
Ventanas | 1.25
(2 rows)

CREATE FOREIGN TABLE
CREATE VIEW
C:\Users\Jeremy Gaibor\Desktop\practica-fragmentacion>docker exec -it pg-campus psql -U admin -d cafeteria -c "SELECT *
FROM clientes_global ORDER BY cliente_id;"
cliente_id | nombre | ciudad | email | telefono
-----
1 | Maria Alvarado | Quevedo | maria@uteq.edu.ec | 0991111111
2 | Luis Cedeno | Babahoyo | luis@uteq.edu.ec | 0992222222
3 | Ana Vera | Quevedo | ana@uteq.edu.ec | 0993333333
4 | Jose Mendoza | Ventanas | jose@uteq.edu.ec | 0994444444
5 | Carla Zambrano | Ventanas | carla@uteq.edu.ec | 0995555555
```

Figura 10: Reconstrucción vertical de `clientes` mediante `clientes_global`.

Ambas vistas devolvieron los mismos resultados que las consultas equivalentes ejecutadas contra la base de datos centralizada del Paso 1, lo que confirma que la fragmentación es transparente para el usuario final.

8. Verificación de las propiedades de fragmentación

Se ejecutaron tres consultas de comprobación (Listado 13) para validar que la fragmentación horizontal cumple las tres propiedades exigidas:

- **Completitud:** la suma de filas de `pedidos` en los tres nodos es igual al total de filas del esquema centralizado.
- **Disjunción:** ningún `pedido_id` se repite entre nodos.
- **Reconstrucción:** la vista `pedidos_global` devuelve exactamente el mismo conjunto de filas que la tabla

centralizada.

```
C:\Users\Jeremy Gaibor\Desktop\practica-fragmentacion>docker exec -i pg-campus psql -U admin -d cafeteria < sql\07_verificacion.sql
filas_globales
-----
      8
(1 row)

 sede | sum
-----
Babahoyo | 4.25
Campus   | 4.25
Ventanas | 2.25
(3 rows)

pedido_id | veces
-----
(8 rows)
```

Figura 11: Verificación de completitud de la fragmentación horizontal.

Como evidencia adicional exigida por la rúbrica, se muestra el conteo de filas de `pedidos` obtenido con `psql` en cada uno de los tres nodos:

```
1 docker exec -it pg-campus psql -U admin -d cafeteria -c "SELECT COUNT(*) FROM pedidos;"
2 docker exec -it pg-babahoyo psql -U admin -d cafeteria -c "SELECT COUNT(*) FROM pedidos;"
3 docker exec -it pg-ventanas psql -U admin -d cafeteria -c "SELECT COUNT(*) FROM pedidos;"
```

```
PS C:\Users\Jeremy Gaibor> docker exec -it pg-campus psql -U admin -d cafeteria -c "SELECT COUNT(*) FROM pedidos;"
count
-----
      3
(1 row)

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug pg-campus
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Jeremy Gaibor> docker exec -it pg-babahoyo psql -U admin -d cafeteria -c "SELECT COUNT(*) FROM pedidos;"
count
-----
      3
(1 row)

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug pg-babahoyo
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Jeremy Gaibor> docker exec -it pg-ventanas psql -U admin -d cafeteria -c "SELECT COUNT(*) FROM pedidos;"
count
-----
      2
(1 row)
```

Figura 12: Conteo de filas de `pedidos` en `pg-campus`, `pg-babahoyo` y `pg-ventanas`.

9. Reflexión

Lo más difícil no fue escribir el SQL de cada fragmento, sino no perder de vista qué fila estaba en qué nodo. Con la fragmentación horizontal de `pedidos` fue fácil: solo hay que filtrar por `sede` y listo, cada nodo se queda con lo suyo. El problema apareció en la fragmentación mixta de `clientes`, donde tocaba cortar por ciudad y además por sensibilidad del dato al mismo tiempo. Ahí sí nos tocó parar y revisar con lápiz y papel qué `cliente_id` iba en cada bloque, porque era fácil dejar a alguien fuera o repetirlo en dos fragmentos por error.

`postgres_fdw` también dio trabajo. La primera vez que intentamos crear el servidor foráneo nos tiró error de conexión, y resultó que el `host` que pusimos en `CREATE SERVER` no era el nombre del contenedor tal como lo definimos en `docker-compose.yml`. Una vez corregido eso, las vistas globales funcionaron sin más problemas y devolvían lo mismo que la base centralizada del Paso 1, lo cual fue satisfactorio de comprobar.

En cuanto a ventajas, lo que más se nota trabajando con esto es que cada sede queda dueña de sus propios datos: si se cae el nodo de Ventanas, Campus y Babahoyo siguen funcionando normal, solo se pierde acceso a los pedidos de esa sede en particular. Eso en un sistema centralizado no pasa. También nos gustó que separar `clientes_publicos` de `clientes_contacto` deja más claro qué datos son sensibles, sin tener que aplicar permisos columna por columna sobre una sola tabla.

Como desventaja, mantener todo esto sincronizado es más trabajo del que parece al principio. Cualquier cambio en el esquema hay que replicarlo a mano en los tres contenedores, y verificar que las tres propiedades (completitud, disjunción, reconstrucción) se sigan cumpliendo no es automático, hay que correr las consultas de comprobación cada vez. Para un sistema pequeño como el de esta práctica es manejable, pero se nota que en un caso real con más tablas y más nodos esto se complicaría bastante.

Referencias

- [1] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*, 4th ed. Cham, Switzerland: Springer Nature, 2020, doi: 10.1007/978-3-030-26253-2.
- [2] PostgreSQL Global Development Group, “postgres_fdw — Foreign Data Wrapper for Remote PostgreSQL Servers,” *PostgreSQL Documentation*. [Online]. Available: <https://www.postgresql.org/docs/current/postgres-fdw.html>
- [3] Docker Inc., “Compose File Reference,” *Docker Documentation*. [Online]. Available: <https://docs.docker.com/compose/compose-file/>